

INF01151 – Sistemas Operacionais II

Aula 06 - Monitores

Prof. Alberto Egon Schaeffer Filho

Introdução

- Exclusão mútua pode ser obtida com variáveis *lock* e semáforos
 - São mecanismos “primitivos” e estão sujeitos a erros no seu uso
 - ***lock/unlock* estão sujeitos a erros de:**
 - » Desatenção na sequência de primitivas *lock/unlock*
 - » Esquecimento de primitiva *lock* ou *unlock*
 - **Semáforos estão sujeitos a erros de:**
 - » Esquecimento de P ou de V
 - » Inicialização
 - » Ordem e quantidade de primitivas
 - Erros levam a *deadlocks* e violação da exclusão mútua

Problemas: utilização incorreta de semáforos

- Erros de programação na utilização de semáforos:

Processo troca a ordem de execução das primitivas:

```
V(s);  
...  
{critical section}  
...  
P(s);
```

Violação da exclusão mútua

Processo substitui chamada de V por P:

```
P(s);  
...  
{critical section}  
...  
P(s);
```

Deadlock

Processo omite chamada a P ou V:

```
P(s);  
...  
{critical section}  
...  
V(s);
```

Violando exclusão mútua ou causando deadlock

Além disso ...

- Variáveis tipo lock
 - Apenas exclusão mútua
- Semáforos
 - Exclusão mútua, sincronização condicional, etc
 - Objetivos diferentes, deveriam ser obtidos c/ programação diferente
 - Dificuldade adicional para escrever e compreender um programa
- Solução: construção de linguagem de mais alto nível

De um modo geral:

- É melhor que **exclusão mútua** seja **implícita**, pois automaticamente evita interferência
- É melhor que **sincronização de condição** seja **explícita**, pois diferentes programas precisam diferentes condições



Monitores

Módulos de um programa que oferecem mais estrutura que semáforos

Monitor

- Monitor pode ser visto como um objeto (ou um módulo, ou um abstract data type) que encapsula:
 - **Variáveis permanentes** representando dados para serem compartilhados
 - Dados internos que armazenam o estado do objeto
 - Privados e acessíveis somente dentro do monitor
 - **Procedimentos** para manipular esses dados (única forma de manipulação)
 - Procedimentos são executados de forma mutuamente exclusiva
 - Exclusão mútua é garantida pelo ambiente (**implicitamente!**)
 - Compilador insere todas as chamadas de sistema e código necessários para criar filas de espera por recursos

Sintaxe e semântica de um monitor

```
monitor name {
```

```
    Declaração de variáveis permanentes
```

```
    Inicialização
```

```
    Procedimentos
```

```
}
```

- Compartilhadas por todos os procedimentos
- Mantém os valores enquanto o monitor existir (estado)

- Procedimentos implementam as operações
- Variáveis locais

- Propriedades de um monitor:
 - Apenas os nomes de procedimentos são visíveis do exterior
 - Procedimentos de um monitor não acessam variáveis externas
 - Variáveis permanentes são inicializadas antes de qualquer procedimento
 - Quando o monitor é criado

Semântica do monitor: exclusão mútua

- No máximo uma instância de um procedimento de um monitor pode estar sendo executada (ativa) em um instante de tempo t
 - Responsabilidade do compilador garantir a exclusão mútua
 - Usa lock e/ou semáforos (uso correto → garantido pelo compilador!)
 - Oferecida **implicitamente** pelo monitor
 - Exclusão mútua
 - Entre chamadas para procedimentos diferentes
 - Entre chamadas para um mesmo procedimentos

Semântica do monitor: variáveis de condição

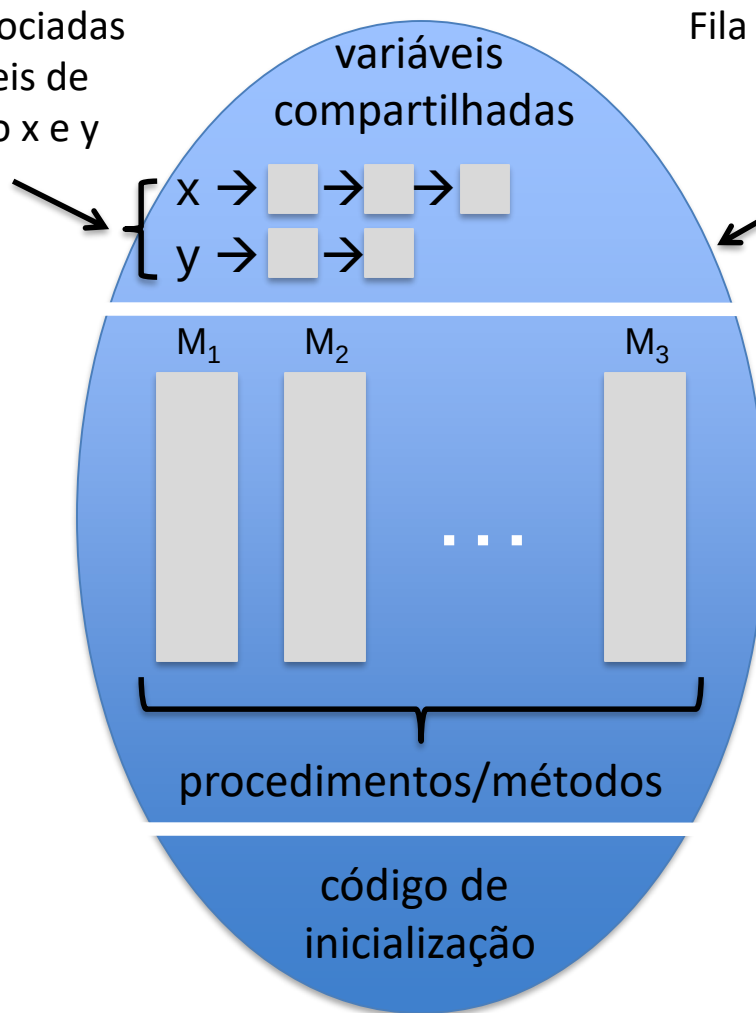
- Variável de condição
 - Sincronização de condição (coordenação de ações)
 - Utilizada para **atrasar um processo** que não pode continuar executando até que o estado do monitor satisfaça alguma condição booleana
 - Também utilizada para **acordar um processo** esperando quando condição se tornar verdadeira
- Variáveis especiais ligadas ao conceito de monitor
 - Declaradas e usadas apenas ***dentro*** do monitor
 - Possuem uma fila associada
 - O “valor” da variável de condição é a fila, mas ela não é vista, nem acessível para o programador
- Só podem ser acessadas via primitivas específicas
 - **wait(cv)** e **signal(cv)**

Primitivas *wait* e *signal*

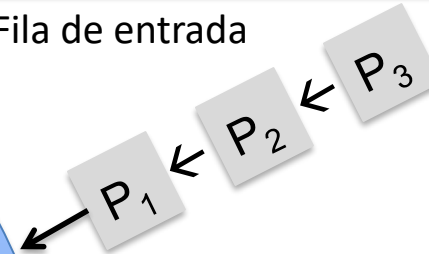
- **`wait (condition_variable)`**
 - Processo que executa **`wait`** bloqueia na variável de condição
 - Retira do monitor, incondicionalmente, o processo que a executa
 - Coloca processo no final da fila associada à variável de condição
- **`signal (condition_variable)`**
 - Acorda um único processo da fila associada à condição sinalizada
 - Se fila estiver vazia no momento do **`signal`**, ele é perdido
 - Não “guarda memória” (diferentemente de semáforos!)
 - As primitivas **`wait`** e **`signal`** implementam uma política FIFO
 - **`wait`** insere o processo no final da fila e **`signal`** acorda o processo da frente da fila
→ comportamento *default*
 - Possível haver outras políticas (e.g., prioridade)

Representação de um monitor

Filas associadas
a variáveis de
condição x e y



Fila de entrada



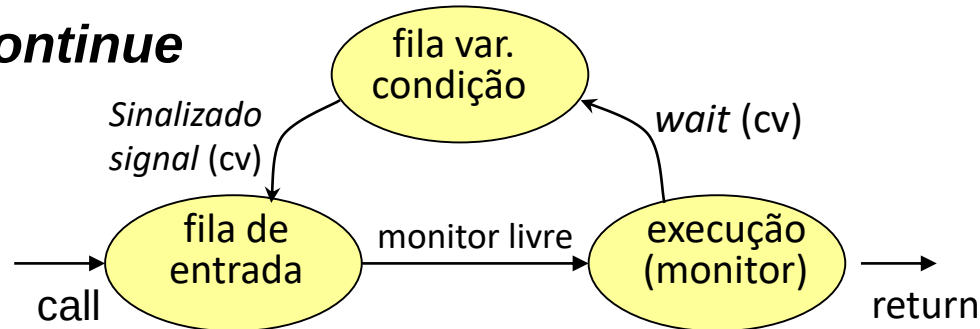
- Uma única fila de espera para entrar no monitor
- Uma fila para cada variável de condição
- Questão é o que fazer com os processos sinalizadores?
 - Semânticas para o **signal**

Estratégias de sinalização

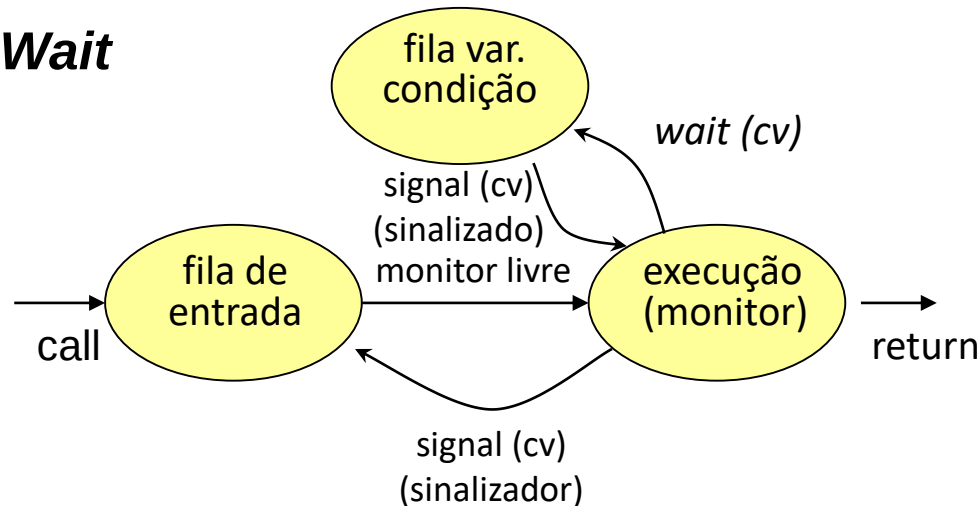
- Quando processo executa **signal**, ele está dentro do monitor e implicitamente tem controle do lock daquele monitor
 - Dilema: se o **signal** irá acordar outro processo, existem agora dois processos que poderiam executar
 - Processo que fez o **signal**? Processo que foi acordado?
- Estratégias de sinalização
 - **Signal and Continue (SC)**: não-preemptiva → Java / ambientes Unix / Pthreads
 - O processo sinalizador continua sua execução e o sinalizado passa para a fila de entrada do monitor
 - **Signal and Wait (SW)**: preemptiva
 - O processo sinalizador vai para a fila de entrada do monitor e o processo sinalizado inicia sua execução
 - **Signal and Urgent Wait (SU)**: preemptiva
 - O processo sinalizador vai para a frente da fila de entrada do monitor e o processo sinalizado inicia sua execução

Diagrama de estados

Signal and Continue



Signal and Wait



Primitivas adicionais para *condition variables* (cv)

- Operações adicionais na fila de variável de condição
 - **wait** e **signal** determinam que processos são acordados na ordem em que foram colocados em espera (FIFO)
 - Como exercer mais controle?
 - **wait(cv, rank)** : processo é posto na fila da cv, mas ordenados de acordo com o valor de *rank* (crescente)
 - **minrank(cv)** : determina rank do processo na frente da fila de cv
 - **empty(cv)** : Retorna *true* se a fila da cv está vazia, senão *false*
 - **signal_all(cv)** : usado se mais de um processo bloqueado puder proceder (ou se o sinalizador não sabe quem deve proceder)
 - Sinaliza todos os processo que estão na fila da cv.
 - Comportamento bem definido para a estratégia SC
 - O que ocorre se a estratégia SW?

Resumo de operações em condition variables

Operação	Descrição
wait(cv)	Espera no final da fila
wait(cv,rank)	Espera em ordem crescente de rank
signal(cv)	Acorda processo da frente da fila e continua
signal_all(cv)	Acorda todos os processos da fila e continua
empty(cv)	True se fila estiver vazia, false caso contrário
minrank(cv)	Valor do rank do processo na frente da fila

Produtor/consumidor com buffer limitado (outra vez...)

- Relação produtor-consumidor
 - Produtor insere mensagens em um buffer compartilhado
 - Consumidor retira mensagens
 - Buffer (*array buf*) de tamanho n contém mensagens que foram inseridas, mas ainda não foram lidas

Produtor:

```
buf[rear] = data;  
rear = (rear+1) % n;
```

Consumidor:

```
result = buf[front];  
front=(front+1) % n;
```

- Com monitor:
 - Exclusão mútua é garantida por definição:
 - Não há necessidade de usar mecanismos para exclusão mútua
 - Dois tipos de sincronização de condição:
 - Buffer não está cheio → possível produzir até completá-lo
 - Buffer não está vazio → possível consumir até esvaziá-lo

Buffer limitado: Monitor

```
monitor Bounded_Buffer {  
    typeT buf[n];  
    int    front = 0, rear = 0, count = 0;  
    cond    not_full,  
           not_empty;  
  
    procedure deposit(typeT data) {  
        while(count == n) wait(not_full);  
        buf[rear] = data; rear = (rear+1) % n; count++;  
        signal(not_empty);  
    }  
    procedure fetch(typeT &result) {  
        while(count == 0) wait(not_empty);  
        result = buf[front]; front = (front+1) % n; count--;  
        signal(not_full);  
    }  
}
```

Leitores e escritores

(agora com monitores!!)

- Dois tipos de processos, leitores e escritores, e uma base de dados
 - Leitores consultam registros e escritores modificam registros
 - Processos escritores devem ter acesso exclusivo a base de dados
 - Processos leitores podem acessar concorrentemente a base de dados
 - Leitores disputam c/ escritores, escritores disputam entre si e c/ leitores
- Condições:
 - Leitores devem esperar até não haver escritores
 - Escritores devem esperar até não haver leitores ou outros escritores
- Questão: posso modelar a base de dados como um monitor?
 - **Não!** Leitores não poderiam acessar monitor concorrentemente
 - Ao invés disso, utiliza monitor apenas para arbitrar acesso à base
 - Processos informam quando querem acessar e quando terminam

Leitores e escritores: Monitor

```
monitor RW_Controller {
    int  nr = 0, nw = 0;
    cond oktoread,          #signaled when nw == 0
        oktowrite;         #signaled when nr == 0 e nw == 0
    procedure request_read( ) {
        while (nw > 0) wait(oktoread);
        nr = nr + 1;
    }
    procedure release_read( ) {
        nr = nr - 1;
        if (nr == 0) signal(oktowrite);
    }
    procedure request_write( ) {
        while (nr > 0 || nw > 0) wait(oktowrite);
        nw = nw + 1;
    }
    procedure release_write( ) {
        nw = nw - 1;
        signal(oktowrite);
        signal_all(oktoread);
    }
}
```

Aninhamento de monitores

- Procedimento de um monitor chama procedimento de outro monitor
 - Lembre-se: processos executam com exclusão mútua dentro do monitor
- Duas estratégias:
 - **closed call**: a exclusão mútua no primeiro monitor é mantida
 - **open call**: o primeiro monitor é liberado e no retorno a ele o processo deve entrar na fila de espera de entrada
- Como de costume, relação custo x benefício

Aninhamento de monitores

- ***closed call***
 - Por definição não há conflitos com variáveis permanentes
 - Serialização de acessos aos monitores reduz concorrência
 - Aumento a possibilidade de dealock (*nested monitor lockout*)
 - Utilizado na implementação de **monitores em Java**
- ***open call***
 - Necessário manter maior controle das variáveis do monitor
 - Maior complexidade

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 5
- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 6 (seção 6.7)

Exercícios

1. Escreva um monitor para simular o comportamento de uma *barreira* de valor N:
 - Monitor deve possuir um único procedimento denominado *gather*, sem argumentos
 - Valor de N é definido por uma variável permanente no próprio monitor

```
monitor Barreira {  
  
    ...  
  
    procedure gather() {  
  
        ...  
  
    }  
  
}
```

Exercícios

2. Escreva um monitor para controlar o acesso a um recurso compartilhado por até N processos concorrentemente:
- Monitor deve ter dois procedimentos *aloca* e *desaloca*, sem argumentos
 - Valor de N é definido por uma variável permanente no próprio monitor

```
monitor Alocador {  
  
    ...  
  
    procedure aloca() {  
        ...  
    }  
    procedure desaloca() {  
        ...  
    }  
  
}
```